

UNIVERSITÉ DU SINE SALOUM EL HADJI IBRAHIMA NIASS

UFR des Sciences Fondamentales et de l'Ingénierie
Département de Mathématiques-Informatique

Travaux Dirigés Programmation Arduino

Thème : Communication I2C et SPI

Enseignant : Dr B. DIOP

Objectif : Maîtriser les protocoles de communication I2C et SPI pour interfacer des capteurs, mémoires et autres périphériques, tout en appliquant des techniques algorithmiques de gestion de données.

Rappel des broches (Arduino Uno) :

- **I2C** : SDA = A4, SCL = A5
- **SPI** : MOSI = 11, MISO = 12, SCK = 13, SS = 10 (par défaut)

Partie I : Communication I2C

Exercice 1. Scanner I2C amélioré avec tableau de périphériques

Le scanner I2C de base détecte les adresses, mais ne garde pas trace des périphériques trouvés. On souhaite créer une version améliorée qui :

- Stocke les adresses détectées dans un tableau.
- Associe un nom à chaque adresse connue (table de correspondance).
- Détecte les changements entre deux scans (périphérique ajouté/retiré).

Travail demandé :

1. Créer un tableau `PROGMEM` associant les adresses I2C courantes à leurs noms (ex: `0x27 = LCD`, `0x68 = RTC DS3231`, `0x76 = BME280`).
2. Écrire une fonction `scanner()` qui remplit un tableau des adresses détectées.
3. Comparer le scan actuel avec le précédent pour afficher les changements.

```
1 #include <Wire.h>
2
3 // Table des peripheriques connus (en Flash)
4 struct PeripheriqueConnu {
```

```

5   byte adresse;
6   const char* nom;
7 };
8
9 const char nom_LCD[] PROGMEM = "LCD I2C";
10 const char nom_RTC[] PROGMEM = "RTC DS3231";
11 const char nom_BME[] PROGMEM = "BME280";
12 const char nom_OLED[] PROGMEM = "OLED SSD1306";
13 const char nom_MPU[] PROGMEM = "MPU6050";
14
15 const PeripheriqueConnu tableConnus[] PROGMEM = {
16     {0x27, nom_LCD}, {0x3C, nom_OLED}, {0x68, nom_RTC},
17     {0x76, nom_BME}, {0x77, nom_BME}, {0x69, nom_MPU}
18 };
19
20 byte adressesPrecedentes[16];
21 byte adressesActuelles[16];
22 byte nbPrecedent = 0, nbActuel = 0;
23
24 void scanner() {
25     nbActuel = 0;
26     for (byte addr = 1; addr < 127 && nbActuel < 16; addr++) {
27         Wire.beginTransmission(addr);
28         if (Wire.endTransmission() == 0) {
29             adressesActuelles[nbActuel++] = addr;
30         }
31     }
32 }
33
34 const char* getNomPeripherique(byte adresse) {
35     // A COMPLETER : chercher dans tableConnus
36 }
37
38 void detecterChangements() {
39     // A COMPLETER : comparer les deux tableaux
40 }

```

Question algorithmique : Quelle est la complexité de la comparaison des deux tableaux ? Comment l'optimiser si les tableaux étaient triés ?

Exercice 2. Communication Maître-Esclave avec protocole structuré

Deux Arduino communiquent en I2C. L'esclave gère plusieurs capteurs et le maître peut demander des données spécifiques via un protocole de commandes.

Protocole défini :

- Commande 0x01 : Lire température (retourne 2 octets : entier signé)
- Commande 0x02 : Lire humidité (retourne 1 octet : 0-100)
- Commande 0x03 : Lire toutes les données (retourne 4 octets)
- Commande 0x10 : Configurer intervalle de mesure (envoie 2 octets)

Travail demandé :

1. Écrire le code de l'Arduino Esclave (adresse 0x08) qui :
 - Reçoit les commandes via `onReceive()`.

- Répond aux demandes via `onRequest()`.
- Stocke la dernière commande reçue pour savoir quoi répondre.

2. Écrire le code de l'Arduino Maître qui interroge cycliquement l'esclave.

```

1 // == CODE ESCLAVE ==
2 #include <Wire.h>
3
4 #define ADRESSE_ESCLAVE 0x08
5
6 volatile byte derniereCommande = 0;
7 int temperature = 0;
8 byte humidite = 0;
9 unsigned int intervalleMesure = 1000;
10
11 void setup() {
12     Wire.begin(ADRESSE_ESCLAVE);
13     Wire.onReceive(receptionCommande);
14     Wire.onRequest(envoiReponse);
15 }
16
17 void receptionCommande(int nbOctets) {
18     if (Wire.available()) {
19         derniereCommande = Wire.read();
20
21         // Si c'est une commande de configuration
22         if (derniereCommande == 0x10 && Wire.available() >= 2) {
23             byte msb = Wire.read();
24             byte lsb = Wire.read();
25             intervalleMesure = (msb << 8) | lsb;
26         }
27     }
28 }
29
30 void envoiReponse() {
31     switch (derniereCommande) {
32         case 0x01: // Temperature
33             Wire.write((temperature >> 8) & 0xFF);
34             Wire.write(temperature & 0xFF);
35             break;
36         case 0x02: // Humidite
37             Wire.write(humidite);
38             break;
39         case 0x03: // Toutes les donnees
40             // A COMPLETER
41             break;
42     }
43 }
44
45 void loop() {
46     // Mise a jour des capteurs
47     static unsigned long derniereMesure = 0;
48     if (millis() - derniereMesure > intervalleMesure) {
49         temperature = analogRead(A0) - 500; // Simule temperature
50         humidite = map(analogRead(A1), 0, 1023, 0, 100);
51         derniereMesure = millis();
52     }
53 }

```

Exercice 3. Réseau I2C multi-esclaves avec collecte de données

Un Arduino Maître collecte les données de 4 Arduino Esclaves (adresses 0x10 à 0x13), stocke l'historique et calcule des statistiques.

Spécifications :

- Chaque esclave renvoie une valeur de capteur sur 2 octets.
- Le maître maintient un buffer circulaire de 10 valeurs par esclave.
- Calcul de la moyenne et détection des esclaves inactifs (timeout).

Travail demandé :

1. Implémenter la structure de données pour stocker l'historique.
2. Écrire la fonction de collecte avec gestion des erreurs I2C.
3. Calculer les statistiques (moyenne, min, max) par esclave.

```
1 #include <Wire.h>
2
3 #define NB_ESCLAVES 4
4 #define TAILLE_HISTORIQUE 10
5
6 struct DonneesEsclave {
7     byte adresse;
8     int historique[TAILLE_HISTORIQUE];
9     byte indexHistorique;
10    byte tailleHistorique;
11    unsigned long dernierContact;
12    bool actif;
13 };
14
15 DonneesEsclave esclaves[NB_ESCLAVES];
16
17 void initialiserEsclaves() {
18     for (int i = 0; i < NB_ESCLAVES; i++) {
19         esclaves[i].adresse = 0x10 + i;
20         esclaves[i].indexHistorique = 0;
21         esclaves[i].tailleHistorique = 0;
22         esclaves[i].actif = false;
23     }
24 }
25
26 bool lireEsclave(int index) {
27     Wire.requestFrom(esclaves[index].adresse, (byte)2);
28
29     if (Wire.available() == 2) {
30         byte msb = Wire.read();
31         byte lsb = Wire.read();
32         int valeur = (msb << 8) | lsb;
33
34         // Ajouter au buffer circulaire
35         esclaves[index].historique[esclaves[index].indexHistorique] =
            valeur;
36         esclaves[index].indexHistorique =
37             (esclaves[index].indexHistorique + 1) % TAILLE_HISTORIQUE;
38     }
```

```

39     if (esclaves[index].tailleHistorique < TAILLE_HISTORIQUE) {
40         esclaves[index].tailleHistorique++;
41     }
42
43     esclaves[index].dernierContact = millis();
44     esclaves[index].actif = true;
45     return true;
46 }
47 return false;
48 }
49
50 float getMoyenne(int index) {
51     // A COMPLETER
52 }
53
54 void verifierTimeouts() {
55     // Marquer inactif si pas de reponse depuis 5 secondes
56     // A COMPLETER
57 }

```

Question : Comment optimiser la mémoire si on avait 20 esclaves au lieu de 4 ?

Exercice 4. Écriture/Lecture EEPROM externe I2C

On utilise une EEPROM I2C (type 24LC256, 32 Ko, adresse 0x50) pour stocker des données de manière permanente.

Spécifications :

- Écriture page par page (une page = 64 octets max pour 24LC256).
- Lecture séquentielle de plusieurs octets.
- Stockage de structures de données (enregistrements de capteurs).

Travail demandé :

1. Écrire les fonctions `ecrireOctet(adresse, valeur)` et `lireOctet(adresse)`.
2. Écrire une fonction `ecrireBloc(adresse, buffer, taille)` qui gère le découpage en pages.
3. Implémenter un système de log circulaire : stocker les 100 derniers enregistrements.

```

1  #include <Wire.h>
2
3  #define EEPROM_ADDR 0x50
4  #define PAGE_SIZE 64
5
6  struct Enregistrement {
7      unsigned long timestamp;
8      int valeur;
9      byte type;
10 }; // 7 octets
11
12 // Adresse de l'index de tete du log circulaire (stockee en adresse 0-1)
13 // Enregistrements stockes a partir de l'adresse 2
14
15 void ecrireOctet(unsigned int adresse, byte valeur) {
16     Wire.beginTransmission(EEPROM_ADDR);

```

```

17 Wire.write((adresse >> 8) & 0xFF); // Adresse haute
18 Wire.write(adresse & 0xFF);       // Adresse basse
19 Wire.write(valeur);
20 Wire.endTransmission();
21 delay(5); // Temps d'écriture de l'EEPROM
22 }
23
24 byte lireOctet(unsigned int adresse) {
25     Wire.beginTransaction(EEPROM_ADDR);
26     Wire.write((adresse >> 8) & 0xFF);
27     Wire.write(adresse & 0xFF);
28     Wire.endTransmission();
29
30     Wire.requestFrom(EEPROM_ADDR, (byte)1);
31     return Wire.read();
32 }
33
34 void ecrireBloc(unsigned int adresse, byte* buffer, int taille) {
35     // A COMPLETER : gerer le decoupage en pages de 64 octets
36     // Attention : ne pas dépasser les limites de page !
37 }
38
39 void ajouterLog(int valeur, byte type) {
40     // Lire l'index de tete actuel
41     // Ecrire l'enregistrement a la bonne adresse
42     // Mettre a jour l'index (circulaire sur 100 enregistrements)
43     // A COMPLETER
44 }

```

Calcul : 100 enregistrements $\times$ 7 octets = 700 octets. À quelle adresse se termine le log ?

Partie II : Communication SPI

Exercice 5. Lecture de capteur SPI avec registres

Un capteur SPI (type accéléromètre) possède plusieurs registres internes accessibles en lecture/écriture.

Protocole du capteur :

- Pour lire : envoyer l'adresse du registre avec bit 7 = 1 (ex: 0x80 | 0x2D).
- Pour écrire : envoyer l'adresse du registre avec bit 7 = 0, puis la valeur.
- Registres : 0x00 = ID (lecture seule), 0x2D = Config, 0x32-0x37 = Données XYZ.

Travail demandé :

1. Écrire les fonctions `lireRegistre(reg)` et `ecrireRegistre(reg, val)`.
2. Écrire une fonction `lireAxes(int* x, int* y, int* z)` qui lit les 6 registres de données en une seule transaction (lecture burst).
3. Vérifier l'ID du capteur au démarrage.

```

1 #include <SPI.h>
2
3 #define CS_PIN 10
4 #define REG_ID 0x00
5 #define REG_CONFIG 0x2D

```

```
6 #define REG_DATA_X0 0x32
7
8 SPISettings configSPI(1000000, MSBFIRST, SPI_MODE3);
9
10 void setup() {
11     pinMode(CS_PIN, OUTPUT);
12     digitalWrite(CS_PIN, HIGH);
13     SPI.begin();
14
15     // Verifier l'ID du capteur
16     byte id = lireRegistre(REG_ID);
17     if (id != 0xE5) {
18         Serial.println(F("Capteur non detecte !"));
19     }
20 }
21
22 byte lireRegistre(byte reg) {
23     byte valeur;
24     SPI.beginTransaction(configSPI);
25     digitalWrite(CS_PIN, LOW);
26     SPI.transfer(reg | 0x80); // Bit 7 = 1 pour lecture
27     valeur = SPI.transfer(0x00); // Envoie dummy, recoit valeur
28     digitalWrite(CS_PIN, HIGH);
29     SPI.endTransaction();
30     return valeur;
31 }
32
33 void ecrireRegistre(byte reg, byte valeur) {
34     SPI.beginTransaction(configSPI);
35     digitalWrite(CS_PIN, LOW);
36     SPI.transfer(reg & 0x7F); // Bit 7 = 0 pour ecriture
37     SPI.transfer(valeur);
38     digitalWrite(CS_PIN, HIGH);
39     SPI.endTransaction();
40 }
41
42 void lireAxes(int* x, int* y, int* z) {
43     byte buffer[6];
44
45     SPI.beginTransaction(configSPI);
46     digitalWrite(CS_PIN, LOW);
47     SPI.transfer(REG_DATA_X0 | 0x80 | 0x40); // 0x40 = auto-increment
48
49     for (int i = 0; i < 6; i++) {
50         buffer[i] = SPI.transfer(0x00);
51     }
52
53     digitalWrite(CS_PIN, HIGH);
54     SPI.endTransaction();
55
56     // Reconstruction des valeurs 16 bits (little-endian)
57     *x = (buffer[1] << 8) | buffer[0];
58     *y = (buffer[3] << 8) | buffer[2];
59     *z = (buffer[5] << 8) | buffer[4];
60 }
```

Exercice 6. Gestion de plusieurs périphériques SPI

On connecte 3 périphériques SPI sur le même bus : un capteur (CS=10), une mémoire Flash (CS=9), et un écran (CS=8). Chacun a des paramètres SPI différents.

Travail demandé :

1. Créer une structure `PeripheriqueSPI` contenant : broche CS, vitesse, mode SPI.
2. Écrire une fonction `selectionner(int index)` qui configure le bon périphérique.
3. Implémenter un système de verrouillage pour éviter les conflits.

```
1 #include <SPI.h>
2
3 struct PeripheriqueSPI {
4     byte pinCS;
5     uint32_t vitesse;
6     byte mode;
7     const char* nom;
8 };
9
10 PeripheriqueSPI peripheriques[] = {
11     {10, 1000000, SPI_MODE0, "Capteur"},
12     {9, 4000000, SPI_MODE0, "Flash"},
13     {8, 8000000, SPI_MODE3, "Ecran"}
14 };
15
16 #define NB_PERIPHERIQUES 3
17 int peripheriqueActif = -1;
18
19 void initialiserSPI() {
20     for (int i = 0; i < NB_PERIPHERIQUES; i++) {
21         pinMode(peripheriques[i].pinCS, OUTPUT);
22         digitalWrite(peripheriques[i].pinCS, HIGH);
23     }
24     SPI.begin();
25 }
26
27 bool selectionner(int index) {
28     if (index < 0 || index >= NB_PERIPHERIQUES) return false;
29     if (peripheriqueActif >= 0) return false; // Deja occupe
30
31     peripheriqueActif = index;
32     SPI.beginTransaction(SPISettings(
33         peripheriques[index].vitesse,
34         MSBFIRST,
35         peripheriques[index].mode
36     ));
37     digitalWrite(peripheriques[index].pinCS, LOW);
38     return true;
39 }
40
41 void liberer() {
42     if (peripheriqueActif >= 0) {
43         digitalWrite(peripheriques[peripheriqueActif].pinCS, HIGH);
44         SPI.endTransaction();
45         peripheriqueActif = -1;
46     }
47 }
```



```

48
49 // Exemple d'utilisation
50 void lireCapteur() {
51     if (selectionner(0)) {
52         byte valeur = SPI.transfer(0x00);
53         liberer();
54     }
55 }

```

Question : Que se passe-t-il si une interruption essaie d'accéder au SPI pendant une transaction ?

Exercice 7. Mémoire Flash SPI avec système de fichiers simplifié

On utilise une mémoire Flash SPI (type W25Q32, 4 Mo) pour stocker des fichiers de données.

Spécifications :

- La Flash s'efface par secteurs de 4 Ko (doit être effacé avant écriture).
- Table d'allocation simple : les 4 premiers Ko stockent un répertoire de fichiers.
- Chaque entrée de répertoire : nom (8 car) + adresse début (3 octets) + taille (2 octets).

Travail demandé :

1. Définir la structure `EntreeFichier` (13 octets).
2. Écrire `creerFichier(nom, donnees, taille)` qui trouve un emplacement libre.
3. Écrire `lireFichier(nom, buffer, tailleMax)` qui retrouve et lit le fichier.

```

1 #define CS_FLASH 9
2 #define CMD_READ 0x03
3 #define CMD_WRITE 0x02
4 #define CMD_ERASE_SECTOR 0x20
5 #define CMD_WRITE_ENABLE 0x06
6
7 struct EntreeFichier {
8     char nom[8];
9     byte adresse[3]; // Adresse sur 24 bits (16 Mo max)
10    uint16_t taille;
11    byte utilise; // 0xFF = libre, 0x01 = utilise
12 }; // 14 octets
13
14 #define MAX_FICHIERS 290 // 4096 / 14 = 292, on garde 290
15 #define DEBUT_DONNEES 4096 // Les donnees commencent au secteur 1
16
17 unsigned long adresseLibre = DEBUT_DONNEES;
18
19 void lireFlash(unsigned long adresse, byte* buffer, int taille) {
20     digitalWrite(CS_FLASH, LOW);
21     SPI.transfer(CMD_READ);
22     SPI.transfer((adresse >> 16) & 0xFF);
23     SPI.transfer((adresse >> 8) & 0xFF);
24     SPI.transfer(adresse & 0xFF);
25
26     for (int i = 0; i < taille; i++) {
27         buffer[i] = SPI.transfer(0x00);
28     }

```

```

29     digitalWrite(CS_FLASH, HIGH);
30 }
31
32 int trouverFichier(const char* nom) {
33     EntreeFichier entree;
34     for (int i = 0; i < MAX_FICHIERS; i++) {
35         lireFlash(i * sizeof(EntreeFichier), (byte*)&entree, sizeof(
entree));
36         if (entree.utilise == 0x01 && strncmp(entree.nom, nom, 8) == 0)
37         {
38             return i;
39         }
40     }
41     return -1; // Non trouve
42 }
43
44 bool creerFichier(const char* nom, byte* donnees, uint16_t taille) {
45     // A COMPLETER :
46     // 1. Verifier que le fichier n'existe pas deja
47     // 2. Trouver une entree libre dans le repertoire
48     // 3. Ecrire les donnees a adresseLibre
49     // 4. Mettre a jour l'entree du repertoire
50     // 5. Mettre a jour adresseLibre
51 }

```

Partie III : Projets Intégrateurs

Exercice 8. Pont I2C-SPI

Créer un système où un Arduino lit des données de capteurs I2C et les stocke sur une mémoire Flash SPI, avec interface de consultation.

Architecture :

- Capteur de température I2C (simulé ou réel).
- Mémoire Flash SPI pour le stockage.
- Enregistrement périodique (toutes les minutes).
- Commandes série pour consulter l'historique.

Travail demandé :

1. Implémenter la lecture du capteur I2C.
2. Implémenter le stockage en log circulaire sur la Flash SPI.
3. Créer une interface série : **LIST** (afficher les N derniers), **EXPORT** (dump complet), **CLEAR** (effacer).

```

1 #include <Wire.h>
2 #include <SPI.h>
3
4 #define CAPTEUR_I2C 0x48
5 #define CS_FLASH 9
6 #define TAILLE_LOG 1000 // 1000 enregistrements max
7

```

```

8 struct Enregistrement {
9     unsigned long timestamp;
10    int16_t temperature; // En centièmes de degrés
11 }; // 6 octets
12
13 uint16_t indexLog = 0;
14 uint16_t nbEnregistrements = 0;
15
16 void enregistrerMesure() {
17     // Lire le capteur I2C
18     Wire.requestFrom(CAPTEUR_I2C, (byte)2);
19     int16_t temp = (Wire.read() << 8) | Wire.read();
20
21     // Préparer l'enregistrement
22     Enregistrement e;
23     e.timestamp = millis() / 1000;
24     e.temperature = temp;
25
26     // Ecrire sur la Flash SPI
27     unsigned long adresse = 4096 + (indexLog * sizeof(Enregistrement));
28     ecrireFlash(adresse, (byte*)&e, sizeof(e));
29
30     // Mettre à jour l'index circulaire
31     indexLog = (indexLog + 1) % TAILLE_LOG;
32     if (nbEnregistrements < TAILLE_LOG) nbEnregistrements++;
33 }
34
35 void traiterCommande(String cmd) {
36     if (cmd.startsWith("LIST")) {
37         int n = cmd.substring(5).toInt();
38         if (n == 0) n = 10;
39         afficherDerniers(n);
40     } else if (cmd == "EXPORT") {
41         exporterTout();
42     } else if (cmd == "CLEAR") {
43         effacerLog();
44     }
45 }

```

Exercice 9. Réseau mixte I2C/SPI avec arbitrage

On crée un système où le bus I2C et le bus SPI doivent cohabiter, avec des accès potentiellement concurrents (interruptions).

Problématique :

- Un timer déclenche une lecture I2C toutes les 100 ms.
- Le programme principal effectue des écritures SPI sporadiques.
- Les deux bus partagent certaines ressources (ex: variables globales).

Travail demandé :

1. Identifier les sections critiques (où les conflits peuvent survenir).
2. Implémenter un mécanisme de verrouillage (mutex simple avec `volatile`).
3. Utiliser `noInterrupts()` / `interrupts()` aux bons endroits.

```
1 #include <Wire.h>
2 #include <SPI.h>
3
4 volatile bool busI2COccupe = false;
5 volatile bool busSPIOccupe = false;
6 volatile int derniereValeurI2C = 0;
7
8 // Buffer partage (section critique !)
9 volatile int bufferPartage[10];
10 volatile byte indexBuffer = 0;
11
12 void setup() {
13     Wire.begin();
14     SPI.begin();
15
16     // Configurer Timer1 pour interruption toutes les 100ms
17     // ... (configuration du timer)
18 }
19
20 // ISR du timer - lecture I2C periodique
21 ISR(TIMER1_COMPA_vect) {
22     if (!busI2COccupe) {
23         busI2COccupe = true;
24
25         Wire.requestFrom(0x48, (byte)2);
26         if (Wire.available() == 2) {
27             int valeur = (Wire.read() << 8) | Wire.read();
28
29             // Section critique : acces au buffer partage
30             noInterrupts();
31             bufferPartage[indexBuffer] = valeur;
32             indexBuffer = (indexBuffer + 1) % 10;
33             interrupts();
34
35             derniereValeurI2C = valeur;
36         }
37
38         busI2COccupe = false;
39     }
40 }
41
42 void ecrireSPI(byte* donnees, int taille) {
43     // Attendre que le bus soit libre
44     while (busSPIOccupe);
45     busSPIOccupe = true;
46
47     noInterrupts(); // Protéger la transaction SPI
48     digitalWrite(CS_PIN, LOW);
49     for (int i = 0; i < taille; i++) {
50         SPI.transfer(donnees[i]);
51     }
52     digitalWrite(CS_PIN, HIGH);
53     interrupts();
54
55     busSPIOccupe = false;
56 }
```

Question : Pourquoi faut-il déclarer les variables partagées avec volatile ?

Exercice 10. Système de monitoring distribué I2C

Concevoir un système complet où un Arduino Maître collecte des données de plusieurs Esclaves spécialisés, les stocke, et les rend accessibles.

Architecture complète :

- Esclave 1 (0x10) : Capteur de température (renvoie 2 octets).
- Esclave 2 (0x11) : Capteur de luminosité (renvoie 2 octets).
- Esclave 3 (0x12) : Compteur d'impulsions (renvoie 4 octets).
- Maître : Collecte, stocke en Flash SPI, interface série.

Travail demandé :

1. Écrire le code générique d'un Esclave (paramétrable par `#define`).
2. Écrire le code du Maître avec :
 - Collecte cyclique avec détection des pannes.
 - Stockage structuré sur Flash SPI.
 - Commandes série : `STATUS`, `READ n`, `STATS`, `RESET`.
3. Calculer le débit de données et la durée avant remplissage de la mémoire.

```
1 // === CODE MAITRE ===
2 #include <Wire.h>
3 #include <SPI.h>
4
5 #define NB_ESCLAVES 3
6
7 struct ConfigEsclave {
8     byte adresse;
9     byte nbOctets;
10    const char* nom;
11    unsigned long derniereLecture;
12    bool actif;
13 };
14
15 ConfigEsclave esclaves[NB_ESCLAVES] = {
16     {0x10, 2, "Temperature", 0, false},
17     {0x11, 2, "Luminosite", 0, false},
18     {0x12, 4, "Compteur", 0, false}
19 };
20
21 struct MesureComplete {
22     unsigned long timestamp;
23     int16_t temperature;
24     uint16_t luminosite;
25     uint32_t compteur;
26     byte masqueActifs; // Bit = 1 si esclave actif
27 }; // 13 octets
28
29 void collecterTout(MesureComplete* mesure) {
30     mesure->timestamp = millis() / 1000;
31     mesure->masqueActifs = 0;
32
33     // Temperature
```

```

34     if (lireEsclave(0, (byte*)&mesure->temperature)) {
35         mesure->masqueActifs |= 0x01;
36     }
37
38     // Luminosite
39     if (lireEsclave(1, (byte*)&mesure->luminosite)) {
40         mesure->masqueActifs |= 0x02;
41     }
42
43     // Compteur
44     if (lireEsclave(2, (byte*)&mesure->compteur)) {
45         mesure->masqueActifs |= 0x04;
46     }
47 }
48
49 bool lireEsclave(int index, byte* buffer) {
50     Wire.requestFrom(esclaves[index].adresse, esclaves[index].nbOctets);
51
52     unsigned long debut = millis();
53     while (Wire.available() < esclaves[index].nbOctets) {
54         if (millis() - debut > 100) { // Timeout 100ms
55             esclaves[index].actif = false;
56             return false;
57         }
58     }
59
60     for (int i = 0; i < esclaves[index].nbOctets; i++) {
61         buffer[i] = Wire.read();
62     }
63
64     esclaves[index].derniereLecture = millis();
65     esclaves[index].actif = true;
66     return true;
67 }
68
69 void afficherStatus() {
70     Serial.println(F("== STATUS =="));
71     for (int i = 0; i < NB_ESCLAVES; i++) {
72         Serial.print(esclaves[i].nom);
73         Serial.print(F(" (0x"));
74         Serial.print(esclaves[i].adresse, HEX);
75         Serial.print(F("): "));
76         Serial.println(esclaves[i].actif ? F("ACTIF") : F("INACTIF"));
77     }
78 }

```

Calcul : Avec une mesure toutes les secondes et 13 octets par mesure, combien de temps peut-on enregistrer sur une Flash de 4 Mo ?

Questions de Synthèse

Question A : Choix du protocole

Pour chaque situation, indiquer si I2C ou SPI est plus adapté et justifier :

1. Connecter 12 capteurs de température sur une même carte.
2. Piloter un écran TFT couleur 320×240 pixels.

3. Lire un accéléromètre à 1000 échantillons/seconde.
4. Ajouter une horloge temps réel (RTC) à un projet existant.
5. Communiquer entre deux Arduino distants de 50 cm.

Question B : Débogage

Un capteur I2C ne répond pas. Lister les vérifications à effectuer dans l'ordre :

1. Vérifications logicielles (code, adresse, bibliothèque).
2. Vérifications matérielles (câblage, alimentation, résistances).
3. Outils de diagnostic (scanner I2C, analyseur logique).

Question C : Optimisation mémoire

On doit stocker les données de 8 capteurs I2C, avec 20 mesures historiques chacun.

1. Calculer la mémoire nécessaire avec des `int` (2 octets par mesure).
2. Proposer une optimisation si les valeurs sont comprises entre 0 et 255.
3. Comment réduire encore la mémoire avec de la compression simple ?

Bon courage !